Ministry of Education, Culture and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 10
ADD YOUR OWN SYSTEM CALL: TRACE

Checked by:
Reiţman Patricia

Chişinău, 2026

Contents

1 Introduction **2**

2 Environment and Initial Verification **2**

3 Part 1: The Number and User Side **3**
3.1 Number, Declaration, Stub, and Disk Image 3
3.2 The User Command 3
3.3 Check 1: Reaching an Unimplemented Kernel Call 3

4 Part 2: Kernel Implementation **4**
4.1 Per-Process State and Handler 4
4.2 Dispatch Table, Names, and Output 4
4.3 Initialization and Inheritance 4
4.4 Check 2: Exact Read Results 5

5 Part 3: Mask Experiments **5**
5.1 All Supported Calls 5
5.2 Fork Only and Inherited Masks 6
5.3 Write Only 7
5.4 Open and Close Only 7
5.5 Additional Wrapper Checks 8

6 Observe Questions and Answers **8**
6.1 Why Only Read Calls with Mask 32? 8
6.2 Why Do Descendants Print Trace Lines? 8
6.3 Why Does a Trace Line Interrupt the Echo Output? 9
6.4 Which Mask Selects Open and Close? 9

7 Results and Committed Deliverable **9**

8 Conclusion **10**

A Recorded Implementation Changes **10**

1 Introduction

This laboratory work extends the xv6 RISC-V teaching operating system with a new system call, `trace`. A process supplies a bit mask selecting which system calls should be printed after execution. Each selected trace line contains the process identifier, the system call name, and its return value.

Lab 9 observed Linux through `strace`; this work implements a tracing mechanism inside the xv6 kernel. The experiment follows syscall number 23 from the user declaration and generated assembly stub to the kernel dispatcher and handler, then tests mask selection and inheritance across process creation.

The report follows the structure of the previous laboratory reports. All terminal listings are captured from the actual VM and QEMU sessions. Excerpts omit build verbosity or unrelated commands only where identified; no screenshots or external assets are required to compile this document.

2 Environment and Initial Verification

A working directory was created at `/home/user/os-lab10`. The existing `/home/user/xv6-riscv` tree contained earlier coursework, so it was copied with its Git history into `/home/user/os-lab10/xv6-riscv`. The original tree was preserved. Existing coursework source changes were recorded in a baseline commit before the Lab 10 implementation, on branch `codex/lab10-trace`.

```
$ bash -c pwd; uname -sr; riscv64-linux-gnu-gcc --version | head -1; qemu-system-
    riscv64 --version | head -1; wc -c README
/home/user/os-lab10/xv6-riscv
Linux 6.14.0-37-generic
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
2441 README
```

Listing 1: Working tree, kernel, RISC-V compiler, emulator, and README size

The baseline built successfully with `make -j2`. A QEMU boot confirmed that the original shell worked before any Lab 10 edits. QEMU was exited with its Ctrl-A, X escape sequence after each checkpoint.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo baseline boot OK
baseline boot OK
$ QEMU: Terminated
```

Listing 2: Initial boot and shell check; compiler output omitted

This version reserves syscall 22 for `sync` and uses `printk` for kernel output, so the new syscall uses number 23 as required by the assignment. The README is 2441 bytes, matching the assignment's expected read sequence.

3 Part 1: The Number and User Side

3.1 Number, Declaration, Stub, and Disk Image

The first stage adds `SYS_trace` in `kernel/syscall.h`, declares `int trace(int);` in `user/user.h`, and adds `entry("trace");` to `user/usys.pl`. The Makefile's `UPROGS` list receives a tab-indented `$U/_trace` entry followed by a continuation backslash, placing the executable in the xv6 filesystem image.

The generated `user/usys.S` stub loads syscall number 23 into register `a7`, executes `ecall`, and returns to the caller. The integer argument is passed in `a0`. The following is an excerpt from the actual stub-inspection output after the completed build:

```
trace:
 li a7, SYS_trace
 ecall
 ret
```

Listing 3: Generated trace stub in user/usys.S

3.2 The User Command

The file `user/trace.c` contains the program supplied by the assignment. It validates the argument count, sets the current process's mask, and executes the requested command. A successful `exec` replaces the program image while retaining the process and its tracing mask.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Listing 4: Complete user/trace.c compiled in the VM

3.3 Check 1: Reaching an Unimplemented Kernel Call

At this checkpoint only the user side was implemented. The tree was rebuilt and booted before continuing to Part 2.

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

Listing 5: Check 1: actual expected failure before the handler is installed

The unknown-call message proves that the program reached the kernel with number 23. The wrapper then saw a negative return value and printed `trace: failed`. This is the intended intermediate result, not the result of the final implementation.

4 Part 2: Kernel Implementation

4.1 Per-Process State and Handler

An integer `trace_mask` was added after the process name in `struct proc`. The new `sys_trace` handler obtains argument zero with `argint`, saves the value on `myproc()`, and returns zero.

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

Listing 6: Handler added to kernel/sysproc.c

4.2 Dispatch Table, Names, and Output

The kernel declares `sys_trace` and maps `SYS_trace` to it in the dispatch table. A table maps all syscall numbers from 1 through 23 to readable names. After the selected handler returns, the dispatcher checks the corresponding mask bit and prints the result already stored in `a0`:

```
p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
  printk("%d: syscall %s -> %ld\n",
         p->pid, syscallnames[num], p->trapframe->a0);
```

Listing 7: Trace logic placed after syscall execution

Placing this code after the handler is necessary to print the actual result. It also means that a syscall which never returns, such as `exit`, does not reach the post-call trace print. The kernel uses `printk` directly; printing a trace line therefore does not recursively invoke a user-level `write` system call.

4.3 Initialization and Inheritance

Newly allocated process state explicitly starts with a zero mask. When a parent forks, the child's mask is copied immediately after its name. In this tree the internal process-creation

function is called `kfork`; it implements the user-visible `fork` syscall.

```
// allocproc(), after setting the process state:
p->trace_mask = 0;

// kfork(), after copying the process name:
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```

Listing 8: Initialization and inheritance of the trace mask

The explicit zero initialization supplements the assignment's edits with the normal initialization of the added field. Inheritance allows descendants to continue tracing. An `exec` does not overwrite this field, so the command launched by the wrapper remains traced.

4.4 Check 2: Exact Read Results

The completed kernel built successfully and booted in QEMU. The required command produced the following actual output:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

Listing 9: Check 2: read-only tracing with the completed kernel

The successful reads total $1023 + 965 + 453 = 2441$ bytes, matching README's size. The final zero return value indicates end of file. Only reads appear because mask 32 is 2^5, and `SYS_read` is 5. The staged verification checked this exact sequence before proceeding to Part 3.

5 Part 3: Mask Experiments

5.1 All Supported Calls

```
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0
```

Listing 10: All supported mask bits enabled

The positive mask $2147483647 = 2^{31} - 1$ enables bits 0 through 30 and therefore all syscall numbers currently used by this tree. The trace includes the `trace` call itself because it installs the mask before the dispatcher checks it. `exec` returns 3 in this xv6

implementation: the new program's argument count. The open returns descriptor 3 and the close returns zero.

5.2 Fork Only and Inherited Masks

The required `forkforkfork` test was executed in a fresh QEMU session. Below is the complete command output for the test, without the emulator startup lines. Process identifiers and scheduling order can vary between executions.

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 3: syscall fork -> 4
4: syscall fork -> 5
5: syscall fork -> 6
5: syscall fork -> 7
6: syscall fork -> 8
6: syscall fork -> 9
7: syscall fork -> 10
5: syscall fork -> 11
6: syscall fork -> 12
6: syscall fork -> 13
5: syscall fork -> 14
5: syscall fork -> 15
6: syscall fork -> 16
6: syscall fork -> 17
6: syscall fork -> 18
5: syscall fork -> 19
6: syscall fork -> 20
5: syscall fork -> 21
6: syscall fork -> 22
6: syscall fork -> 23
6: syscall fork -> 24
5: syscall fork -> 25
6: syscall fork -> 26
5: syscall fork -> 27
5: syscall fork -> 28
6: syscall fork -> 29
6: syscall fork -> 30
5: syscall fork -> 31
6: syscall fork -> 32
5: syscall fork -> 33
5: syscall fork -> 34
5: syscall fork -> 35
5: syscall fork -> 36
6: syscall fork -> 37
5: syscall fork -> 38
6: syscall fork -> 39
6: syscall fork -> 40
6: syscall fork -> 41
5: syscall fork -> 42
6: syscall fork -> 43
6: syscall fork -> 44
6: syscall fork -> 45
5: syscall fork -> 46
6: syscall fork -> 47
6: syscall fork -> 48
6: syscall fork -> 49
```

```
47: syscall fork -> 50
5: syscall fork -> 51
5: syscall fork -> 52
6: syscall fork -> 53
5: syscall fork -> 54
48: syscall fork -> 55
5: syscall fork -> 56
6: syscall fork -> 57
5: syscall fork -> 58
6: syscall fork -> 59
5: syscall fork -> 60
6: syscall fork -> 61
21: syscall fork -> 62
5: syscall fork -> 63
6: syscall fork -> 64
5: syscall fork -> -1
6: syscall fork -> -1
OK
ALL TESTS PASSED
```

Listing 11: Fork-only tracing and the required test result

Multiple processes print fork results because children inherit the mask. The negative return value `-1` represents a failed process allocation when resources are exhausted. This is an expected part of the stress test. The test completed with `OK` and `ALL TESTS PASSED`.

```
$ echo after-fork-test
after-fork-test
```

Listing 12: The ordinary shell remains usable after the fork test

5.3 Write Only

```
$ trace 65536 echo hi
hi4: syscall write -> 2

4: syscall write -> 1
```

Listing 13: Write-only tracing, preserving the observed output interleaving

Mask $65536 = 2^{16}$ selects `SYS_write`. The xv6 `echo` program writes the two bytes of `hi` first, then writes a separate one-byte newline. The kernel prints the first trace line immediately after the first write returns, before `echo` issues its newline write. This is why the trace text follows `hi` on the same line.

5.4 Open and Close Only

To select both calls, bit 15 and bit 21 must be set:

$$(1 \ll 15) \mid (1 \ll 21) = 32768 + 2097152 = 2129920.$$

```
$ trace 2129920 grep hello README
5: syscall open -> 3
```

```
5: syscall close -> 0
```

Listing 14: Experimental verification of the open-and-close mask

5.5 Additional Wrapper Checks

A zero mask suppressed trace lines while allowing the command to run. Missing arguments displayed the usage message, and an unavailable command produced the wrapper's execution-error message.

```
$ trace 0 echo untraced
untraced
```

Listing 15: Additional command-line check

```
$ trace
usage: trace mask command [args...]
```

Listing 16: Additional command-line check

```
$ trace 32 missingcommand
trace: exec missingcommand failed
```

Listing 17: Additional command-line check

6 Observe Questions and Answers

The answers below are also saved in both `report.md` and the assignment's requested `notes.txt` under `/home/user/os-lab10`.

6.1 Why Only Read Calls with Mask 32?

Question: In `trace 32 grep hello README`, why do you see only `read`, and not `open` or `close`?

Answer: The binary representation of 32 has only bit 5 set. That bit selects syscall 5, `read`. The bits for `open` (15) and `close` (21) are clear. Those operations still execute; their trace lines are suppressed.

6.2 Why Do Descendants Print Trace Lines?

Question: In `forkforkfork`, different process identifiers print lines. Which line from Part 2 makes that possible?

Answer: The assignment's inheritance line is `np->trace_mask = p->trace_mask;`. It copies the parent's mask into the child during process creation, so the child's own fork calls are also traced.

6.3 Why Does a Trace Line Interrupt the Echo Output?

Question: Why is the trace line between `hi` and its newline? How many times does `echo` call `write`?

Answer: For `echo hi`, there are two writes: one of two bytes for `hi` and one of one byte for the newline. The kernel prints each trace line after its corresponding write returns. The first trace therefore appears before the program has written its newline.

6.4 Which Mask Selects Open and Close?

Question: Which mask traces `open` (15) and `close` (21), and nothing else?

Answer: `2129920`, the bitwise OR of `1 « 15` and `1 « 21`. The experiment above confirms that it prints only an open and a close for the tested command.

7 Results and Committed Deliverable

The initial tree, user-side checkpoint, and final kernel each built and booted. Both required checkpoints matched the expected behavior. The fork-inheritance test passed, and all three required mask experiments were completed. Additional checks verified the combined open/close mask, zero-mask behavior, and wrapper errors. `git diff -check` reported no whitespace problems.

The completed implementation is committed in the VM working tree on branch `codex/lab10-trace`. The following output records the final commit and changed files:

```
$ bash -c git log -2 --oneline; git status --short; git show --stat --oneline HEAD
4f95820 Add per-process trace system call and mask inheritance
3952f44 Preserve existing coursework before Lab 10
?? args
4f95820 Add per-process trace system call and mask inheritance
 Makefile         |  1 +
 kernel/proc.c    |  2 ++
 kernel/proc.h    |  1 +
 kernel/syscall.c | 31 ++++++++++++++++++++++++++++++
 kernel/syscall.h |  1 +
 kernel/sysproc.c |  9 +++++++++
 user/trace.c     | 19 +++++++++++++++++++
 user/user.h      |  1 +
 user/usys.pl     |  1 +
 9 files changed, 66 insertions(+)
```

Listing 18: Recorded Git history, working-tree status, and implementation summary

The untracked `args` file is a pre-existing compiled program copied from the earlier coursework tree; it is not part of the Lab 10 implementation commit. All nine implementation files, including the new `user/trace.c`, are committed. A portable Git bundle containing the complete history and this branch was saved as `xv6-lab10.bundle` in the VM lab directory and copied to the host's `Reports/source_code_and_materials_of_all_labs/os-lab10` folder. The bundle was verified successfully on the host. It can be restored with:

```
git clone -b codex/lab10-trace xv6-lab10.bundle xv6-riscv
```

Listing 19: Restoring the committed xv6 source from the portable Git bundle

This restoration command is provided for future use; it is not a recorded test session. The original VM working tree remains ready for `make qemu`.

8 Conclusion

The new `trace` system call connects a user-level request to a kernel handler through a generated RISC-V stub and the syscall dispatch table. Per-process mask state selects the events to print, persists across `exec`, and is inherited by children. The successful read sequence, fork stress test, and write interleaving demonstrate that the implementation behaves as required. The work also illustrates why tracing must occur after a handler returns and why syscall return values and process scheduling affect the observed output.

A Recorded Implementation Changes

The following is the actual diff against the preserved pre-Lab-10 coursework baseline for the eight existing files. The complete new `user/trace.c` file is reproduced in Part 1. Together they document all nine implementation files.

```
$ git diff -- Makefile kernel/proc.h kernel/proc.c kernel/sysproc.c kernel/syscall.h
    kernel/syscall.c user/user.h user/usys.pl
diff --git a/Makefile b/Makefile
index 3accbe6..2813348 100644
--- a/Makefile
+++ b/Makefile
@@ -133,6 +133,7 @@ UPROGS=\
    $U/_cat\
    $U/_find\
    $U/_zombie\
+   $U/_trace\
    $U/_run\
    $U/_pingpong\
    $U/_echo\
diff --git a/kernel/proc.c b/kernel/proc.c
index 31f23a3..2dd5d14 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -124,6 +124,7 @@ allocproc(void)
 found:
   p->pid = allocpid();
   p->state = USED;
+  p->trace_mask = 0;

   // Allocate a trapframe page.
   if ((p->trapframe = (struct trapframe *)kalloc()) == 0) {
@@ -288,6 +289,7 @@ kfork(void)
   np->cwd = idup(p->cwd);

   safestrcpy(np->name, p->name, sizeof(p->name));
+  np->trace_mask = p->trace_mask;

   pid = np->pid;

diff --git a/kernel/proc.h b/kernel/proc.h
index 5603697..121d8ad 100644
```

```
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -101,4 +101,5 @@ struct proc {
   struct file *ofile[NOFILE];  // Open files
   struct inode *cwd;           // Current directory
   char name[16];               // Process name (debugging)
+  int trace_mask;              // Bit N selects syscall N
 };
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 1800f20..8df9614 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -103,6 +103,7 @@ extern uint64 sys_link(void);
 extern uint64 sys_mkdir(void);
 extern uint64 sys_close(void);
 extern uint64 sys_sync(void);
+extern uint64 sys_trace(void);

 // An array mapping syscall numbers from syscall.h
 // to the function that handles the system call.
@@ -130,9 +131,36 @@ static uint64 (*syscalls[])(void) = {
   [SYS_mkdir]   = sys_mkdir,
   [SYS_close]   = sys_close,
   [SYS_sync]    = sys_sync,
+  [SYS_trace]   = sys_trace,
   // clang-format on
 };

+static char *syscallnames[] = {
+  [SYS_fork] = "fork",
+  [SYS_exit] = "exit",
+  [SYS_wait] = "wait",
+  [SYS_pipe] = "pipe",
+  [SYS_read] = "read",
+  [SYS_kill] = "kill",
+  [SYS_exec] = "exec",
+  [SYS_fstat] = "fstat",
+  [SYS_chdir] = "chdir",
+  [SYS_dup] = "dup",
+  [SYS_getpid] = "getpid",
+  [SYS_sbrk] = "sbrk",
+  [SYS_pause] = "pause",
+  [SYS_uptime] = "uptime",
+  [SYS_open] = "open",
+  [SYS_write] = "write",
+  [SYS_mknod] = "mknod",
+  [SYS_unlink] = "unlink",
+  [SYS_link] = "link",
+  [SYS_mkdir] = "mkdir",
+  [SYS_close] = "close",
+  [SYS_sync] = "sync",
+  [SYS_trace] = "trace",
+};
+
 void
 syscall(void)
 {
@@ -144,6 +172,9 @@ syscall(void)
```

```
     // Use num to lookup the system call function for num, call it,
     // and store its return value in p->trapframe->a0
     p->trapframe->a0 = syscalls[num]();
+    if ((p->trace_mask >> num) & 1)
+      printk("%d: syscall %s -> %ld\n",
+             p->pid, syscallnames[num], p->trapframe->a0);
   } else {
     printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
     p->trapframe->a0 = -1;
diff --git a/kernel/syscall.h b/kernel/syscall.h
index 5a4a39f..f66fb28 100644
--- a/kernel/syscall.h
+++ b/kernel/syscall.h
@@ -21,3 +21,4 @@
 #define SYS_mkdir  20
 #define SYS_close  21
 #define SYS_sync   22
+#define SYS_trace  23
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index cd25557..f3b5d40 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
@@ -110,3 +110,12 @@ sys_uptime(void)
   release(&tickslock);
   return xticks;
 }
+
+uint64
+sys_trace(void)
+{
+  int mask;
+  argint(0, &mask);
+  myproc()->trace_mask = mask;
+  return 0;
+}
diff --git a/user/user.h b/user/user.h
index e571a5e..66ad64a 100644
--- a/user/user.h
+++ b/user/user.h
@@ -26,6 +26,7 @@ char *sys_sbrk(int, int);
 int pause(int);
 int uptime(void);
 int sync(void);
+int trace(int);

 // ulib.c
 int stat(const char *, struct stat *);
diff --git a/user/usys.pl b/user/usys.pl
index 4d054e8..f9185b5 100755
--- a/user/usys.pl
+++ b/user/usys.pl
@@ -43,3 +43,4 @@ entry("sbrk");
 entry("pause");
 entry("uptime");
 entry("sync");
+entry("trace");
```

Listing 20: Actual changes to existing xv6 files